MMA3001 Project Documentation Report: Estimating composite stiffness degradation from ultrasonic signals

Abstract

This project investigates whether simple ultrasonic features can estimate the published stiffness-degradation index of a carbon-fibre specimen excluded from model training. A public NASA fatigue dataset provided labels for only three specimens. After auditing labels and repeated recordings, mean-centred RMS and peak-to-peak amplitude were normalised against each specimen's first usable reference. Four regression families were compared on L1S11 and L1S12. Protocol v2 selected single-path linear regression using the lowest worst-direction validation MAE; the rule preceded new v2 scores, but earlier v1 single-path scores were known. The selected model was refitted on 25 Layup 1 checkpoints and tested on eight L2S11 checkpoints from another layup. Its MAE was 0.0374 and R^2 was −2.27. A post hoc zero predictor achieved slightly lower MAE, 0.0362. The tested pipeline therefore did not demonstrate useful transfer to L2S11. The contribution is a reproducible feasibility assessment, not an operational structural-health-monitoring tool.

1. Engineering problem and intended use

Carbon-fibre composite parts in aircraft can lose stiffness under repeated loading (fatigue). This damage is often not visible from the outside. Direct stiffness assessment requires suitable load–deformation measurements, which can be difficult to obtain during service. Small piezoelectric (PZT) sensors bonded to the part can send ultrasonic waves through it under suitable acquisition conditions. If changes in these waves follow the loss of stiffness, the sensors could monitor the part in service. This project asks a narrow version of that question. Can two simple, normalised amplitude features estimate the published stiffness-degradation index of a specimen that the model was not trained on? [1,2]

The intended use is a feasibility check, not an inspection or remaining-useful-life tool. Its output estimates a published index, not crack size, delamination area or Young's modulus directly, and must not guide continued-service decisions. The scope is limited to three labelled specimens because extra recordings, paths and frequencies cannot replace independent labelled specimens.

The self-selected project and its dataset were approved in person by teaching staff (Keenan Granland) during a practical class.

2. Data, inputs and outputs

The NASA dataset contains fatigue tests on carbon-fibre coupons with three laminate layups, with logbooks, PZT recordings, strain records and X-ray images. The audit found 1,495 MATLAB PZT files but only 39 distinct specimen–cycle label candidates, all belonging to L1S11, L1S12 and L2S11. These include baseline zeros and one negative value. The project reused these published values and it did not independently measure or revalidate stiffness at those 39 checkpoints. Other specimens lacked suitable scalar labels. Tables 1 and 2 define the terms and input/output contract. [1–3]

Table 1. Key terms.

Term	Meaning in this report
Specimen / coupon	One test piece. For example, L1S11 is Layup 1, specimen 11.
Cycle	One load and unload in the fatigue test.
Checkpoint	A stop in the test at a set cycle count, where recordings are made. One checkpoint has one label.
Path	One sending sensor (actuator 1–6) and one receiving sensor (7–12). There are 36 paths.
Label (index)	The published stiffness-degradation value at a checkpoint. Its exact physical definition is unconfirmed. Zero values require context because some are missing-data placeholders.
Feature	A number calculated from a signal (here RMS or peak-to-peak), divided by the same number at the reference.
Reference	The earliest usable checkpoint of a specimen. Its features are the divisors.
MAE	Mean absolute error: the average size of the prediction error, in index units.

Table 2. Inputs and outputs of the workflow.

Item	Format and meaning	Units, limits and handling
PZT measurements	Actuator and receiver signals in .mat files. Amplitude units unknown.	Each signal used has 2,000 samples, all finite and not constant. No conversion to volts or energy.
Acquisition metadata	Specimen, cycle count, path, nominal frequency, gain, drive setting, test condition and load.	Cycles are counts. Frequency is in kHz. Load is in kips (1 kip ≈ 4.45 kN). The code reads these fields from the file.
Regression target	Published stiffness-degradation index, one number per checkpoint.	Missing or placeholder labels are left out and never filled in. The exact physical definition is not documented.
Feature table	CSV file. One row per specimen and checkpoint, with the reference used.	RMS and peak-to-peak ratios, with no units. Repeat recordings are combined by their median. Paths are not extra samples.
Predictions and metrics	CSV and JSON files, and plots.	Predictions, MAE and RMSE are in index units. R^2 has no units. Predictions are not clipped.

Selecting recordings and references

The main pipeline uses one path, actuator 1 to receiver 7 (path 1→7), at 150 kHz. It uses the full signal of 2,000 samples. It uses only recordings made under load (the “loaded” condition). For Layup 1, the load must be between 3.95 and 4.05 kips. The gain and drive settings must also match the usual values of the specimen: 33 and 65 for L1S11, and 35 and 50 for L1S12. The reference of each specimen is its earliest usable checkpoint, which is cycle 1. The project does not assume that the reference is undamaged, and it does not use the label of the reference. L1S11 has 14 usable checkpoints and L1S12 has 13. Without the references, 13 and 12 checkpoints remain for modelling. [3]

L2S11 has a different layup and higher test load. Its metadata-based load window was 4.53–4.63 kips, and its earliest eligible reference was cycle 3. Cycle 1 was excluded by the acquisition rules because it was part of a

static load ramp; its negative label was therefore not scored. The eight scored checkpoints were 10, 1,000, 10,000, 20,000, 30,000, 40,000, 50,000 and 60,000 cycles. [3]

The code never repairs bad data. If a recording breaks a rule, the code excludes it and records the reason. The function `eligibility_reasons` in `scripts/protocol_v2.py` returns the list of rules that each recording fails. Missing labels stay missing. Two different labels at one checkpoint, or a reference value of zero or less, stop the run with an error. The code records a hash of each source file when it reads the file and checks the hash again before scoring. The tests in `tests/test_protocol_v2.py` check these rules.

3. Computational methods and assumptions

Signal features and aggregation

Each received signal s has N = 2,000 samples. The code first removes the mean of the signal. It then calculates two features: the root-mean-square (RMS) amplitude, and the peak-to-peak (PTP) amplitude, which is the maximum minus the minimum. It divides each feature by the same feature of the reference signal r. This gives two ratios, f_1 and f_2, with no units. The function is `waveform_features` in `scripts/common.py`.

$$x_i = s_i - mean(s)$$
$$RMS(s) = \sqrt{[(1/N)\,\Sigma_i\, x_i{}^2]}$$
$$PTP(s) = max(s) - min(s)$$
$$f_1 = RMS(s)/RMS(r), \quad f_2 = PTP(s)/PTP(r)$$

These features count samples, not seconds, so they do not need the sampling rate. Section 8 explains why the stored sampling rate is not reliable. Division by the reference cancels a common multiplicative gain if it is unchanged between the reference and measurement. It does not remove other effects, for example a change in sensor bonding or a difference between specimens. A checkpoint often has two or more repeat recordings, and the code uses their median. The independent unit is the specimen, not the recording, because the checkpoints of one specimen share its fatigue history. [3]

Protocol v2 compared the original single-path features with an alternative using all 36 paths at 150 kHz. For each path, RMS and peak-to-peak were divided by that path's reference values. Taking the median across paths then produced two aggregate features per checkpoint. This changed the inputs but not the number of independent samples. Metadata and remapping checks supported consistent path identification, without proving the physical sensor mapping independently. [3]

Regression models and alternatives

Ordinary least squares (OLS) linear regression finds the linear fit with the smallest sum of squared errors. For two features, the model is $\hat{y} = \beta_0 + \beta_1 f_1 + \beta_2 f_2$. A second, independent solver, based on QR factorisation, gave the same fit from the saved features. QR does not form and invert the normal-equation matrix, which is less stable. The other models come from scikit-learn [7]. Prediction outside the joint training-feature region involves extrapolation, whose behaviour depends on the model. [3,4]

The baseline least-squares implementation uses `numpy.linalg.lstsq`. Independent QR and scikit-learn fits provide numerical cross-checks. The recorded checks use a relative tolerance of 10^{-9} and include matrix-rank checks. Reported design-matrix condition numbers were 15–69; these matrices were not close to singular, but fitted coefficients can still change when inputs are perturbed or rescaled (Week 9 §9.1.1 [4]). The ten MLP fits used L-BFGS and converged in 22–61 iterations without convergence warnings. [3]

Table 3. Models compared and the reason for each.

Method	Purpose and trade-off
Linear regression	Few parameters and easy to interpret. Can predict beyond the training range. Assumes one linear link that holds for every specimen.
Decision tree	Can learn thresholds. With few checkpoints, splits can be unstable. Outside the training input range, predictions remain terminal-leaf averages rather than extending a trend.
RBF support-vector regression	Can learn smooth curves. Depends on input scaling and on fixed settings.
Small MLP	Can learn curves and interactions. Results change with the random seed. Five fixed seeds are reported, and none was picked as the best.
Log-cycle OLS	Baseline. Uses only $\log_{10}$ of the cycle count. Shows whether the sensor features add any information.
Training-mean predictor	Baseline. Always predicts the mean training label. Uses no signal.

The comparison settings were fixed in `config/model_comparison.json` before the new model-comparison scores, and protocol v2 used them unchanged. No settings were tuned because using evaluation scores for tuning would make that specimen part of model development. OLS and log-cycle OLS use an intercept. The decision tree has a maximum depth of 2, at least 3 checkpoints per leaf and at least 6 checkpoints to split. The SVR uses a radial-basis kernel with $C = 1.0$, $\varepsilon = 0.1$ and $\gamma = 0.5$. The MLP has one hidden layer of 3 tanh units and uses L-BFGS with $\alpha = 1.0$ and up to 2,000 iterations. It runs with seeds 7, 23, 42, 71 and 101, and the median result is reported. Linear, SVR and MLP inputs are standardised inside a scikit-learn pipeline. SVR and MLP also standardise the target. Every scaler learns only from the training specimen. [3]

4. Validation design and verification

Two separate questions

This project asks two questions, and they must stay separate. First, does the code calculate what it should? This is our verification. Tests with known answers, the second QR solver, a comparison with scikit-learn and leakage tests answer it. Second, does the model predict well for a specimen that it has not seen? This is our validation. Passing tests answers only the first question. It does not show that the model is physically valid or that it generalises. [3]

Development and final test

L1S11 and L1S12 were the development pair. Each model was trained on one specimen and scored on the other. Then the roles were swapped. A random split of recordings was not used. One label is copied into many files, so a random split would put the same checkpoint in training and in testing. Scalers learn only from the training specimen. The test specimen uses its own reference signal as its divisor, and this uses none of its labels. A real system would therefore need one early reference recording of each new part.

Protocol v2 chose the model with the smallest worst-direction MAE, which is the larger of the two MAEs from swapping the specimens. If two models were within 0.0005, the rule chose the simpler model, then the single-path features. This rule was committed to git before any new v2 score. But the single-path scores from v1 were known when the rule was written. So the selection is not fully prospective. [3]

After selection, the linear model was frozen. It was refitted on all 25 non-reference Layup 1 checkpoints. It was then scored once on L2S11. L2S11 was kept out of model development. But its labels were seen during the early data audit, so it is not completely untouched. L2S11 is also a different layup. With one test specimen, the project cannot separate a layup effect from a specimen effect.

Metrics

$$MAE = (1/n)\,\Sigma_i\,|y_i - \hat{y}_i|$$
$$RMSE = \sqrt{[(1/n)\,\Sigma_i\,(y_i - \hat{y}_i)^2]}$$
$$R^2 = 1 - \Sigma_i\,(y_i - \hat{y}_i)^2 \,/\, \Sigma_i\,(y_i - mean(y))^2$$

MAE was the selection metric. It gives the typical size of the error, in index units. RMSE gives more weight to large errors. R^2 compares the squared error with the error of always predicting the mean of the test set. A negative R^2 means that the model is worse than that mean. No application-specific allowable inspection error was established in this project. So the results cannot show that the method is fit for real inspection.

The later recordings of L1S11, after 100,000 cycles, have no labels. They were used only for three plausibility checks that were written in advance and so they cannot give an accuracy. The zero-prediction comparison and the other diagnostics were added after the test (post hoc) and did not change the frozen model.

All tests pass. The command `python -m pytest` runs 100 tests and 78 subtests in 10.9 s. The HTML test report is `results/test_report/report.html`. [3]

5. Results

Table 4 shows the first (v1) comparison, with two features from path 1→7. The best model depended on the direction. Linear regression was best from L1S11 to L1S12. The small MLP was best in the reverse direction. The MLP values are the median of five seeds. [3]

Table 4. Version 1 comparison on Layup 1: MAE / RMSE for each training direction, in index units.

Model	L1S11 → L1S12 MAE / RMSE	L1S12 → L1S11 MAE / RMSE
Linear regression	0.050532 / 0.058254	0.029357 / 0.035001
Decision tree	0.105158 / 0.117024	0.045803 / 0.051352
RBF SVR	0.111193 / 0.126674	0.056257 / 0.064136
Small MLP	0.078193 / 0.087694	0.020942 / 0.023866
Log-cycle OLS	0.090299 / 0.105431	0.092013 / 0.098381
Training-mean predictor	0.107731 / 0.119134	0.083110 / 0.086881

In protocol v2, linear regression on path 1→7 had a worst-direction MAE of 0.0505. The next best were the single-path MLP (0.0782), the 36-path MLP (0.0888) and 36-path linear regression (0.0903). The 36-path MLP had a better reverse-direction MAE (0.0113), but a worse worst direction. So the rule selected single-path linear regression. This does not show that path 1→7 or 150 kHz is the best physical choice. It only compares the feature sets that were tested. Table 5 gives the one-time test on L2S11. [3]

Table 5. One-time test on L2S11 (eight checkpoints, index units).

L2S11 predictor	MAE	RMSE	Interpretation
Frozen linear model	0.0374	0.0442	R^2 = −2.27; no demonstrated transfer
Fixed log-cycle control	0.0946	0.1014	Carries a different training-target relationship
Fixed training-mean control	0.1015	0.1044	Carries Layup 1's higher target level
Constant zero — post hoc	0.0362	0.0437	Comparable to the fitted model with n = 8; computed after the test

On L2S11, the frozen model outperformed both planned controls, which carried Layup 1's higher target level. However, the post hoc zero predictor had slightly lower MAE (0.0362 versus 0.0374) and RMSE (0.0437 versus 0.0442). Eight correlated checkpoints from one specimen therefore cannot establish statistical equivalence, but also provides no evidence of an advantage for the fitted model. Rank agreement was also weak (post hoc Spearman correlation 0.19). The result therefore does not demonstrate useful transfer to L2S11. Figure 1 shows the predictions. [3]

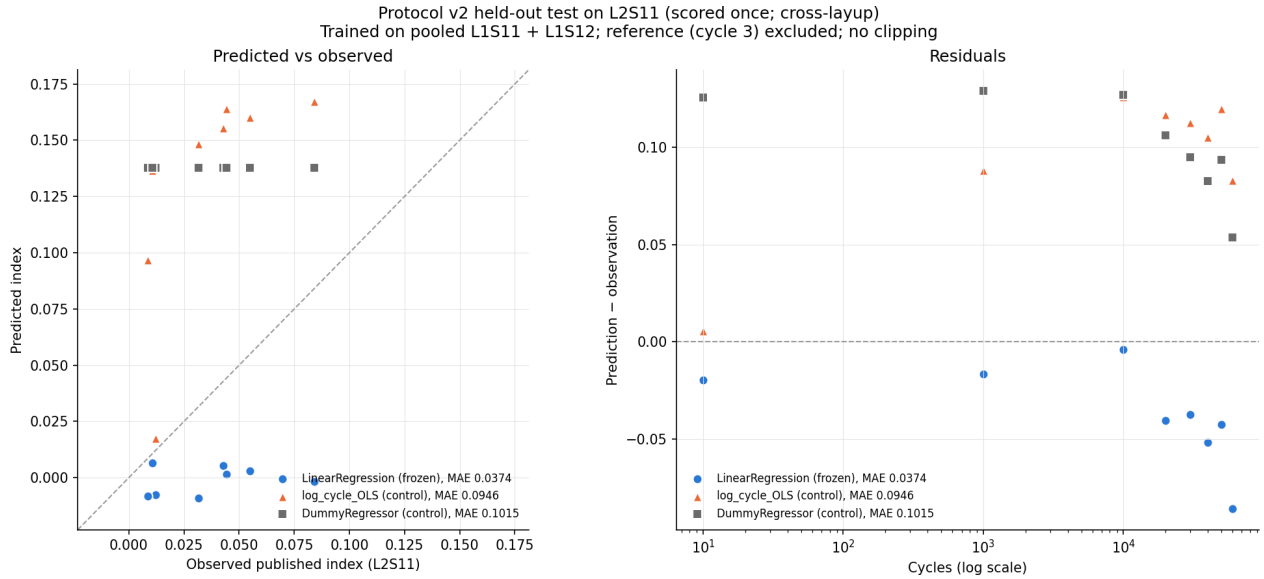


Figure 1. Protocol v2 held-out test on L2S11 (n = 8 scored checkpoints; single-path 1→7 features; index units). Left: predicted against observed published index for the frozen linear model and the two predeclared controls; the dashed line is perfect agreement. Right: residuals against cycle count. Every frozen-model residual is negative. The post hoc zero predictor (MAE 0.0362) is not plotted. Source: `results/protocol_v2/test_L2S11/predictions_L2S11.png`. [3]

6. Robustness and limits

Choice of repeat recording

Most checkpoints have two or more repeat recordings. The main analysis uses their median. To test this choice, the analysis was repeated with the first recording and with the last recording. The references, labels, splits and models did not change. Table 6 shows the MAE of the two-feature OLS model. [3]

Table 6. MAE of two-feature OLS for each choice of repeat recording (index units).

Repeat treatment	L1S11 → L1S12	L1S12 → L1S11
Median	0.050532	0.029357
First	0.050592	0.037760
Last	0.052183	0.033315

The reverse direction changed more. With the last repeats, a model with RMS only gave an MAE of 0.031399, slightly better than the two-feature model. A later check of the logbook found timestamps that go backwards, so "first" and "last" may be swapped at L1S11 80,000 cycles. The saved results are kept, with this caveat. These are robustness checks, not independent repeats of the experiment.

Feature coverage and late-cycle check

The convex hull is the smallest region that contains all training feature points. All 12 L1S12 test points were outside the L1S11 hull. In the reverse direction, 11 of 13 were outside; only 10,000 and 20,000 cycles were inside. All 16 usable L2S11 checkpoints were outside the pooled Layup 1 hull, and 8 of them had labels. So, most predictions were extrapolations. Though, a point outside the hull does not prove that its prediction is wrong though. [3]

The late-cycle L1S11 check failed its first criterion. The check assumed non-decreasing damage-related stiffness loss as a plausibility criterion, rather than a verified property of the published index. But the mean prediction from 125,000 to 227,000 cycles was 0.083, which is below the prediction of 0.114 at 100,000 cycles. The drop occurs at about 177,000 cycles. At that point, the amplitude of all 36 paths increases together. A change in sensor bonding or mounting is a possible cause, but no record confirms it. With no labels, this check cannot measure the late-cycle error.

Change from the original traction-free plan

An earlier feasibility study [2] chose recordings made with no load (traction-free). This avoids effects from the grips and the load of the test machine. A written rule, based on metadata only, chose 450 kHz and paths 1→10, 2→8 and 2→11. That plan kept all 29 labelled Layup 1 checkpoints and their undamaged references. The implemented regression used loaded recordings at 150 kHz and path 1→7 instead. Unfortunately, no record was kept explaining this change. The archived walkthrough, `archive/walkthrough/walkthrough.txt`, used a loaded 150 kHz path 1→7 recording as its example, and later scripts kept that set-up. Nobody compared it with the feasibility plan. [2,3]

The 36-path extension also used loaded recordings at 150 kHz. So, the project never tested the traction-free plan, and the failure on L2S11 does not show how that plan would perform. Path 1→7 also shows an early peak at an almost fixed time in every specimen. This may be electrical crosstalk, which does not travel through the material and so does not change with damage. If so, it can dominate full-signal amplitude features. This remains a hypothesis.

7. Performance and reproducible implementation

Runtime and optimisation

A profiler (cProfile) showed that most of the time goes to decoding the MATLAB files, not to the calculations. The fix reads up to four files at the same time and keeps their order (`iter_coupons` in `scripts/raw_loading.py`). This cut the median runtime from 12.77 s to 7.77 s: 39% less, or 1.64 times faster. The outputs were byte-identical. This change makes the workflow faster. It does not change the accuracy. Table 7 summarises the performance results. [3]

Table 7. Runtime and memory, before and after optimisation.

Characteristic	Original or baseline	Improved or selected
Raw-workflow median runtime	12.77 s	7.77 s with four-thread prefetch
Numerical output	Recorded baseline	Byte-identical in the reported timing comparison
Peak traced allocation	9.51 MiB peak traced, one worker (median of 3)	Not measured
Model fit and predict cost	All six v1 models: 186 kB peak traced	Selected linear model: 85 kB peak traced

Each timing used a new process, with one warm-up and three alternating measured pairs per mode. Operating-system caches were not cleared. Each run included imports, feature extraction, hash checks, modelling and plots for the 254 Layup 1 files. The runs used Windows 11, Python 3.13.3, NumPy 2.3.0, SciPy 1.17.1 and scikit-learn 1.9.0, on a laptop with an AMD Ryzen 7 7735HS processor and 16 GB of DDR5 RAM. The times come from this one machine. The reproducible result is that both modes give identical outputs. [3]

Peak memory was measured with Python's tracemalloc, in new processes, three times per task (scripts/memory_profile.py, results/memory_profile/). Feature extraction for the 254 files peaked at 9.51 MiB. Fitting and predicting with the selected linear model peaked at 85 kB. All six v1 models together peaked at 186 kB. These values count only the memory that tracemalloc tracks, not the total memory of the process. The four-thread mode was not measured. Worker count alone does not establish a bound on retained decoded files.

Feature extraction scales approximately with traces × samples. With two predictors and 25 pooled training checkpoints, fitting contributed little to the measured runtime. Profiling therefore justified prioritising file

decoding over regression arithmetic. The measured speedup applies to this workload and hardware; it does not establish a general hardware limit or measured arithmetic intensity. [3,4]

Repository and reproducibility

The repository keeps code (`scripts/`), tests (`tests/`), settings (`config/`), saved results (`results/`), documents (`docs/`) and early work (`archive/`) in separate folders. The shared logic is in `common.py`, `raw_loading.py` and `protocol_v2.py`. The README explains the set-up and how to run each step. `requirements-lock.txt` records the exact package versions. pdoc builds HTML documentation from the docstrings (`docs/html/index.html`). Reruns write to `results/rebuilt/`, so they cannot overwrite saved results. [3]

Verification includes an independent refit of the regression, tests with known answers, tests that show that test labels cannot leak into training and checks that source files did not change. A rerun on a separate Linux installation gave the same error values as the saved results. The code licence (MIT) does not cover the NASA data or third-party material.

The repository is at https://github.com/iali0008/mma3001-composites-fatigue.

8. Interpretation and conclusions

What the evidence shows

The project gives a reproducible calculation and comparison of simple amplitude features on a small dataset with imperfect labels. Within Layup 1, the sensor-based models were better than the cycle-count and mean baselines. But the best model depended on the direction, the choice of repeat recording changed the results, and most test points were outside the training range. On L2S11, the model was no better than predicting zero.

This conclusion applies only to what was tested: full-signal amplitude features, this reference rule, loaded recordings, 150 kHz and these specimens. It does not show that ultrasonic monitoring fails in general, and it does not show that all normalised amplitude features fail, or that the traction-free plan would fail. The data also cannot support remaining-life predictions or reliability claims for a population of parts.

Physical and statistical limits

The labels were copied from published stiffness tables into the data files. This project did not recalculate them. The index is probably the normalised stiffness loss, $1 - E/E_0$, but there was no documentation that confirms this. So all results are in index units, not in percent of stiffness loss. The converted strain files have no load data, so the labels cannot be checked independently. [1,3]

The files store a sampling rate of 1.2 MHz. The conversion script writes this value as a fixed constant. It does not agree with the excitation frequencies: one 150 kHz cycle spans 80 samples, which implies about 12 MHz. This is strong evidence for about 12 MHz, but it is not a calibration. The amplitude features do not use time, so this does not affect the results. Time-of-flight or energy features would need the correct rate. The amplitude units are also unknown.

Only two development specimens and one external test specimen were used with suitable labels. The checkpoints of one specimen are linked, so to treat each recording or path as a separate example would overstate the evidence. The L1S12 label jumps from 0.084 to 0.233 between 10,000 and 30,000 cycles, and its 20,000-cycle label is missing. The cause is unknown. Label errors may limit any model. The reference method needs an early recording of each part, so it cannot make sure that waves travel in the same way in different layups.

Lessons and future work

The main lesson is that verified code and a faster workflow cannot compensate for an inadequately justified measurement choice. Future work should first justify the acquisition setup, then investigate features that isolate the transmitted wave. Physical timing features require a verified sampling rate, while geometry-based interpretation requires verified sensor locations. These were not independently established in this project; that does not mean the dataset contains no sensor-layout information.

More labelled specimens would help more than more paths. The raw machine files of the unlabelled specimens could give new labels after decoding and calibration. A test of the traction-free plan is justified. But the L2S11 results are now known, so such a test must be labelled exploratory. In conclusion, the tested pipeline did not demonstrate useful transfer to L2S11. Its value is a clear test of feasibility, its failure modes, and what a stronger study needs.

9. AI use and critical reflection

AI tools were used to develop the project and help prepare this report. Table 8 gives full disclosure based on what the brief requires, and the reflection after is written on my own.

Table 8. Disclosure of AI use

Required disclosure	Disclosure
Tools and versions	ChatGPT/Codex (OpenAI), 22–25 September 2026, and ChatGPT for the first draft of this report (October 2026). Claude Code in VS Code and Claude on claude.ai (Anthropic), 11–24 October 2026.
Tasks assisted	Codex: audit, feasibility study, walkthrough, v1 code and repository. Claude Code: v2, tests, fixes, documentation and profiling. Claude on claude.ai: AI-assisted review, explanations, presentation drafts and checking the report. ChatGPT: initial report draft.
Approximate contribution	I made or approved the scientific decisions, and I am responsible for them: the label-audit rule, the specimen-level split, the selection rule fixed before scoring, the no-tuning rule, the single test on L2S11 and the zero-prediction comparison added after that test. I checked every number in this report against a saved results file. I wrote the conclusions, the limits and the reflection. AI tools wrote most of the first code (v1 with Codex; protocol v2, the tests and the profiling with Claude Code) and the first draft of this report (ChatGPT). A fair summary is that the design and the checks are mine, and the implementation and drafting were heavily assisted.
Reason for using AI	I used it to learn the MATLAB (.mat) file format and to map the dataset. I used it to write and debug the pipeline and the tests within the project timeline. I used it for an independent second review of the code and the claims. I used it draft layout of the project report. AI proposed some methodological options and drafted parts of the protocol; I reviewed and approved the final decisions and remain responsible for them.
Verification	I ran the tests, compared OLS with QR and scikit-learn, checked saved result values and derived the feature equations. I confirmed the hash guard after changing a source file and reproduced error values on Linux. Leakage tests passed, although I did not deliberately break them. I reviewed behaviour and main functions, but not every generated line of code.
Student responsibility	I chose the scope and obtained approval, approved specimen-level splits and the selection rule, avoided tuning on evaluation results, and retained the post hoc zero baseline. I narrowed unsupported conclusions and excluded figures lacking saved evidence.
Effect on learning	AI helped me understand specimen-level validation and training-only scaling. However, an example configuration became the design without an explained change from the traction-free plan. This taught me to check measurement assumptions and record decisions before implementation.

Errors and limitations of assistance

The most important error was a change of set-up with no recorded reason. The project moved from a traction-free 450 kHz plan to a loaded 150 kHz set-up. The example in an AI-written walkthrough became the design without a comparison. This was found in the final review. It shows that code can be reproducible and tested and still rest on a poorly justified assumption.

Two AI-assisted code reviews, by Codex and by Claude, were compared with each other. They found three more AI errors. First, AI-written project notes gave figures for repeat spread and reference sensitivity with no source file. Review finding R9 (`docs/review/independent_review.md`) flagged them, and they are not cited. Second, a Claude chat summary described the multi-path result wrongly, although the saved results were correct. Since then, every number in this report comes from a results file (finding C9,

`docs/review/claude_ai_review.md`). Third, an early conclusion claimed more than the evidence showed. It was narrowed to the tested model and specimen (commit `56320f5`).

Personal reflection and ownership

Most of the important decisions were mine. I chose the project and had it approved. I set the specimen as the unit of evidence, not the recording. I chose the leave-one-specimen-out design. I approved the selection rule, which Claude Code drafted, and it was committed before the v2 scores. I did not tune on the two specimens. I kept L2S11 for one external test. I also decided to report the zero-prediction comparison, although it weakened the result, because the report must show what the evidence supports. I can explain the method without AI: the feature equations, the OLS fit and its QR check, the scaler limits, and why the L2S11 result means that the model did no better than predicting zero (MAE 0.0374 against 0.0362), while its R^2 of −2.27 means that it was worse than predicting the mean of the test labels. One choice needed a second look. I first accepted the loaded 150 kHz path 1→7 example from the walkthrough without a reason for the change from the traction-free 450 kHz plan. In the final review I found no record of the reason, and I made that gap a main limitation of the report. If I started again, I would keep a dated decision record, and I would require a written physical reason before changing a measurement setup. I would treat an AI-written example as a proposal to check, not as a default.

I am responsible for the scientific content of this report. I directed the project, made the design and reporting decisions above, and checked every reported number against a saved results file. I ran the tests, compared the OLS fit with the independent QR solver, confirmed once that the hash check stops the run after a source file changes, and reran the code on a separate Linux installation with the same error values. I checked the references against their sources and removed AI-written claims that had no results file behind them. I did not review every line of AI-written code. My checks were at the level of behaviour, tests, saved outputs and close reading of the main functions: `waveform_features`, `eligibility_reasons` and the protocol v2 scoring code. I state that limit here rather than claim more. I will answer the presentation questions without AI, as the brief requires [5].

ChatGPT wrote the first draft of this report from the project brief, the project records and the saved results. Claude checked every number against the saved result files, added the measured values and rewrote some parts of the text in plain English (ASD-STE100 style), to make the report easier to digest. The judgements, conclusions and personal reflection are mine.

References

[1] A. Saxena, K. Goebel, C. C. Larrosa and F.-K. Chang, “CFRP Composites Data Set,” NASA Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA. Download: https://phm-datasets.s3.amazonaws.com/NASA/2.+Composites.zip (accessed 11 October 2026). No licence specified; acknowledgement of the data donors requested. Label provenance and the stored sampling rate: Technical Report, Appendix A.

[2] “L1S11 / L1S12 PZT feature feasibility,” internal project analysis prepared with Codex, 22 September 2026, data/pzt_feasibility/ASSESSMENT.md in [3]. Not an independent published study.

[3] Ibrahim Saneeh, “mma3001-composites-fatigue,” GitHub repository, code commit 8fefaba, 2026. https://github.com/iali0008/mma3001-composites-fatigue. Supporting records: docs/progress.md, docs/protocol_v2.md, docs/methodology.md, results/model_comparison/, results/protocol_v2/, results/repeat_sensitivity/ and results/memory_profile/.

[4] Monash University, MMA3001 Numerical Methods and Machine Learning, 2026 notebooks: Week 1 §1.3 (documentation, pdoc) and §1.5 (testing, pytest); Week 2 (profiling, timing, FLOPs and arithmetic intensity); Week 3 (interpolation and extrapolation); Week 5 §5.6 (grouped splits and safe validation order); Week 9 §9.1.1 (numerical stability, forward and backward error).

[5] Monash University. MMA3001 Numerical Methods and Machine Learning Project Brief, 2026, pp. 4–8 and 10–15. Used to organise report content and check rubric coverage.

[6] A. Saxena, K. Goebel, C. C. Larrosa, V. Janapati, S. Roy and F.-K. Chang, “Accelerated aging experiments for prognostics of damage growth in composite materials,” Proc. 8th International Workshop on Structural Health Monitoring, Stanford, CA, 2011.

[7] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.